

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Professional Skills for Engineers (CO2001)

Assignment

“Recommender System”

Instructor(s): Nguyễn Cao Trí

Students: Nguyễn Thái Học - 2311100 (*Group TN01 - Team 01, **Leader***)
Huỳnh Đức Nhân - 2312420 (*Group TN01 - Team 01*)
Phạm Lê Tiến Đạt - 2310687 (*Group TN01 - Team 01*)
Phạm Trần Minh Trí - 2313622 (*Group TN01 - Team 01*)

HO CHI MINH CITY, DECEMBER 2022



Contents

List of Symbols	3
List of Acronyms	3
List of Figures	5
List of Tables	5
Member list & Workload	5
1 Main concepts about Recommender System	6
1.1 Recommender System and its operation	6
1.2 Types of Recommender system	6
1.2.1 Simple Recommender	7
1.2.2 Content-based Recommender	7
1.2.3 Collaborative Recommender	8
1.2.4 Hybrid Recommender	9
1.3 Use cases	9
1.4 Challenges faced	9
2 Recommender System with Netflix [2]	11
2.1 Introduction	11
2.2 System Architecture	11
2.2.1 Personalized Video Ranker (PVR)	12
2.2.2 Top-N Video Ranker	12
2.2.3 Trending Now	12
2.2.4 Continue Watching	12
2.2.5 Video-to-Video Similarity (Sims)	12
2.2.6 Page Generation and Diversity	13
2.2.7 Evidence Selection	13
2.2.8 Search-Driven Recommendations	13
2.3 Continuous Improvement through Experimentation	13
2.3.1 A/B Testing: The Gold Standard	13
2.3.2 Choosing the Right Metrics	14
2.3.3 Offline Experimentation: Faster, But Riskier	14
2.3.4 Interpreting A/B Test Results	14
2.3.5 Testing New vs. Existing Users	14
2.3.6 Speeding Up Innovation	14
2.4 Conclusions	15
3 Algorithms demonstration	16
3.1 Content-based filtering: Term Frequency - Inverse Document Frequency (TF-IDF)	16
3.2 Collaborative filtering: Singular Value Decomposition (SVD)	17
4 Algorithm implementation and result	19
4.1 Term Frequency - Inverse Document Frequency Model Implement	19
4.2 Singular Value Decomposition Model Implement	21
4.3 Evaluate TF-IDF and SVD model	24



References

27



List of Symbols

$\mathbb{N}$ Set of natural numbers

$\mathbb{R}$ Set of real numbers

$\mathbb{R}^+$ Set of positive real numbers

List of Acronyms

ODE (First-Order) Ordinary Differential Equation

IVP Initial-Value Problem

LTE Local Truncation Error

DS Dynamical System

Fig. Figure

Tab. Table

Sys. System of Equations

Eq. Equation

e.g. For Example

i.e. That Is



List of Figures

1	Types of Recommender System	6
2	Content-based Filtering Recommender System	7
3	Collaborative Filtering Recommender System	8
4	Netflix on website	11
5	Trending now on Netflix	12
6	Cosine similarity	16
7	Matrix factorization	17
8	SVD in linear algebra, visualized	18
9	Recommendation for user 164 using TF-IDF	21
10	Recommendation for user 164 using SVD	23
11	Evaluation of SVD model	24

List of Tables

1	Member list & workload	5
---	----------------------------------	---



Member list & Workload

No.	Fullname	Student ID	Problems	% done
1	Nguyễn Thái Học	2311100	Main concepts about Recommender System	100%
2	Phạm Lê Tiến Đạt	2310687	Recommender System & Algorithm of Netflix	100%
3	Phạm Trần Minh Trí	2313622	Algorithms Demonstration TF-IDF and SVD	100%
4	Huỳnh Đức Nhân	2312420	Algorithms Implementation TF-IDF and SVD	100%

Table 1: Member list & workload

1 Main concepts about Recommender System

1.1 Recommender System and its operation

Recommender System, also called recommender, is an AI system that integrates Data Analytics and Machine Learning algorithms to evaluate and understand user behaviors and interests. Additionally, The amount of items is huge, so having these insights helps the system utilizing metadata effectively to suggest items that users may want to use in the future.

Depending on the operation of the system, it will extract features about user behaviors or the items used via user reactions (love, like, dislike, hate, comments), purchases, user effort (clicks, search, time, events), ... etc. Then, it will compare these information with other users or items to find out the groups that they belong to.

Whenever users in the group consume more items, these items will be suggested to other users. Otherwise, whenever a new product with the same category with the products bought is announced, the user will know its existence on many platforms.

1.2 Types of Recommender system

A recommender system consists of 2 main components: item (the product recommended to users) and query (user and context information). For example, in Netflix, items are films, while query contains user's preferences, watched films and rating about them.

Then those items and queries are mapped to vectors that forms an embedding space. Whenever the system want to make recommendation, it will get a subsets of them through candidate generation (using similarity measures like cosine, dot product or Euclidean distance), scoring and re-ranking them to find out the appropriate recommendations.

The data used for recommendation is divided into 2 types: implicit and explicit. The explicit ones are user's preferences, ranking, clicks, ... etc. They are quantitative data and have the weight for each. While the implicit ones are comments, searches, ... etc, those are qualitative data that need a stage to analyze the meaning of them by using NLP.

The characteristics of the corpus we are working with will determine the model that we should choose. Here are some kinds of it:

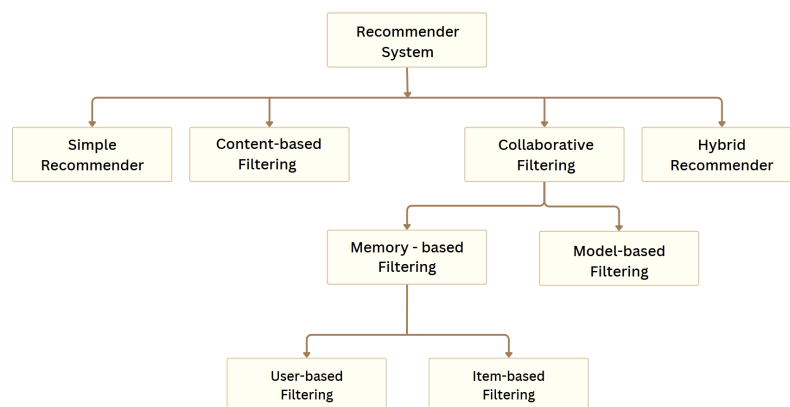


Figure 1: Types of Recommender System

1.2.1 Simple Recommender

Simple recommender suggests items that have high score or ranking from the community due to the fact that common users like common things. For example, Suggesting IMDB top 250 films, or hot new feeds to users. It is easy to implement, but it has a huge disadvantage is that the recommendation is irrelevant to some users, or the system can not explore new items since having high score takes time.

1.2.2 Content-based Recommender

About Content-based recommender, it collects user's preferences and used items. Then it considers main features from them, the list of items that have these main features will be suggested to user. For example, when a user used item A, he will be suggested to use item A1, A2, A3, neither B nor C. The principle is when user like a specific item, it is likely that he want to get similar ones.

About its construction, Content-based filtering systems represent items and users as vectors in a vector space according to item's features and user's preferences. Proximity is used to determine the similarity between items or between user's preference and items. The closer 2 vectors are in space, the more similar they're considered to be. Vectors similar to previous items according to their supplied features will be recommended to the user.

Content-based Filtering

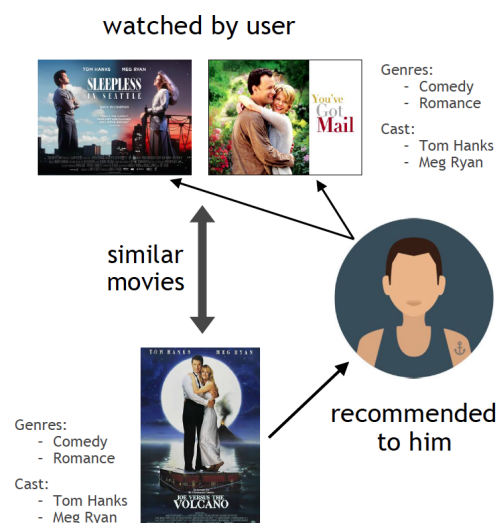


Figure 2: Content-based Filtering Recommender System

1. Advantages

- Recommended items often meet user's needs
- If the items has been categorized correctly, this can be beneficial hugely for the system.
- It does not need other user's data. So it can be used for large scale datasets

2. Disadvantages

- (a) Hard to explore new items, since it rely much on item used by users. It somehow lose user's interest.
- (b) An item is not categorized yet or been categorized incorrectly due to personal choice.

1.2.3 Collaborative Recommender

About Collaborative filtering Recommender, it consider user's interactions with items, and compares it to other user's interactions to find out the group that the user belongs to. Whenever a new product is used by members in the group, it will be suggested to that user. The principle is that users that share same interest may want to use same items. Some big tech company use those types of recommender system: Amazon, Spotify

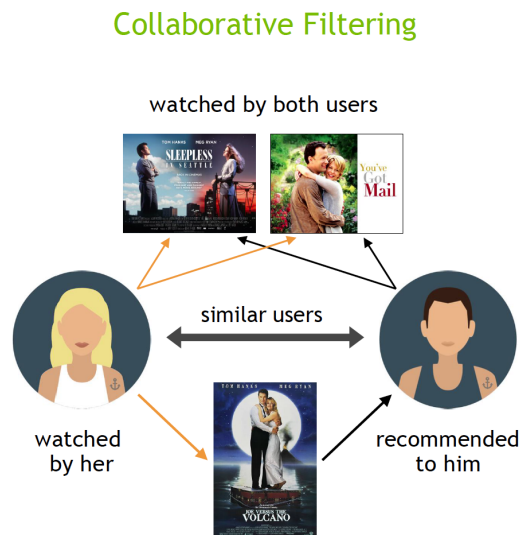


Figure 3: Collaborative Filtering Recommender System

1. Advantages

- (a) Latent feature: not need to get the understanding of item features. So It does not need the domain knowledge.
- (b) Discover more user's interest

2. Disadvantages

- (a) Cold-start Problem: If the system does not contains much historical data about user interactions, it hard to give suggestion to them.
- (b) Sparsity Matrix: Small number of active users give responds to a subset of the whole dataset. Even some popular items get a few of ratings.
- (c) Use other technique like Deep learning, Clustering, Association, Bayesian networks to fix the matrix multiplication's limitations like cold-start problem, but they are hard to implement and run on large scale of data and users.

There are a lot of variances of those types of recommender systems. For example, one that applied the concepts of Collaborative Filtering but it not give recommendations according to all users in the group. Instead, it finds out one or some top common user in the group and hold them as being delegates for that group.

1.2.4 Hybrid Recommender

When the system does not have item features to categorize them, it will use the Collaborative Filtering. Otherwise, when the system does not contains user's interactions to items, it is hard to find out group for that user, it use the Content-based.

1.3 Use cases

In digital world rapidly growing like today, technology has altered the way people interacts with others, so businesses need to approach their customer in a new way. Emerging technology like AI, high computing architectures, ... allows businesses to process and analyze user data in real time, leading to more accurately personalized recommendations.

Both users and businesses can take advantage of the application. Personalized recommendation saves great amount of time for users, avoiding scrolling endlessly to find out the right items to consume. Sometimes, after users give bunch of responds to the system, it can even suggest items that the user have not thought to look for on their own, opening new approaches or give more useful items, which can not do via traditional system.

On the business's side, Giving the right choice for customers can increase customer's purchases, calculated by time or money or invitations to other customers. If they raise a good marketing campaign using this system, it can raise their profit exponentially, more attractive than traditional methods. Due to some research, companies that have applied this method increase their revenue up to 15 percent. In long term, the greater amount of data about user references, the more precise of the recommendation becomes, leading to occur more positive responds from users. It is like a loop help company grow strongly.

Recommender system has been applied in many fields in real life. These are common applications:

1. Entertainment and media: Suggesting videos on YouTube, newsfeed and friends on Facebook, or even partners on dating platforms.
2. E-commerce and retail: Using demographic characteristics and geographic to determine user's class like Amazons or Shopee
3. Travel and hospitality: Suggesting hotel and accommodation options, restaurants, activities and experiences according to a person's budget and travel history.
4. Entrepreneur like marketing, IT, Consultant, ...

1.4 Challenges faced

Even though Recommender systems offer significant opportunities, businesses face several common challenges when implementing them: [4]

1. Cost and Complexity: These systems require complex architectures and substantial computing resources to store, process, and analyze massive amounts of data in real time.



2. **Reliable Big Data:** A large volume of high-quality data is essential for accuracy. The more data a system has, the better its recommendations become.
3. **Bias and Irrelevant Recommendations:** A lack of historical data or an abundance of intentionally misleading user interactions can lead to inaccurate or biased recommendations.

Additionally, users can experience negative effects when using recommendation-based applications improperly. These include cognitive issues like "brain rot," an excessive dopamine response due to frequent short-term gratification, and unrealistic expectations from constantly seeing others' achievements.

2 Recommender System with Netflix [2]

2.1 Introduction

In the era of big data and digital content overload, recommender systems play a vital role in personalizing user experiences. One of the most prominent and successful examples of a large-scale recommender system is Netflix—a leading global streaming platform for movies and TV shows[1]. With millions of users and an ever-growing content library, Netflix faces a significant challenge: how to recommend the right content to the right user at the right time.

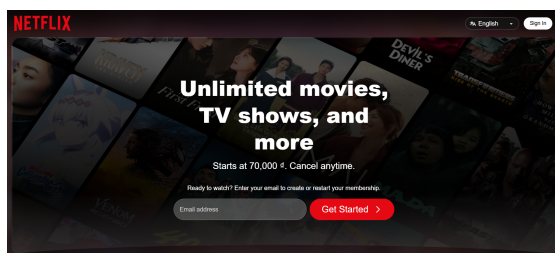


Figure 4: Netflix on website

Netflix has heavily invested in the development of sophisticated recommendation algorithms, ranging from traditional collaborative filtering techniques to advanced deep learning-based models. The goal goes beyond simply increasing watch time; it's about enhancing user satisfaction, engagement, and long-term retention. Building such a system requires more than just analyzing user history—it involves understanding individual preferences, behavior patterns, and contextual factors.

In this section, we will explore how Netflix designs, implements, and continuously improves its recommendation system. From the high-level architecture to the core algorithms, we'll examine the technical foundations and strategies that make Netflix's recommender system one of the most effective in the industry. This insight not only highlights the importance of personalization in modern digital platforms but also offers valuable lessons for applying recommender systems in various other domains.

2.2 System Architecture

The architecture of Netflix's recommender system is one of the most complex and refined examples of large-scale personalization in the world. Rather than relying on a single model or algorithm, Netflix utilizes a collection of specialized recommendation algorithms, each designed to handle a specific aspect of the user experience. These components work together to create a seamless, engaging, and personalized interface—especially on the Netflix homepage, which is the main gateway for user interaction.

At the heart of Netflix's recommendation engine is the homepage as the Recommendation Hub, which is presented every time a user logs in. This page typically consists of rows of videos, each curated based on different algorithms. Each row represents a specific use case or theme—for instance, genres, trends, similarities to recently watched content, or content that the user has not finished watching. The layout is non-static and fully personalized, meaning that no two users are likely to see the same content in the same order.

Core Algorithms includes:

2.2.1 Personalized Video Ranker (PVR)

PVR is responsible for generating genre-based rows such as “Suspenseful Movies” or “Romantic Comedies.” This algorithm works by ranking the entire video catalog (or subsets like genres) for each user. It balances personalized preferences with popularity signals. Since PVR is widely used across various rows, it must produce general-purpose rankings that are reasonably effective for a broad range of use cases.

2.2.2 Top-N Video Ranker

Unlike PVR, which must rank a large portion of the catalog, the Top-N Ranker focuses on optimizing the very top results. It is used to populate the “Top Picks for You” row. This algorithm is optimized using metrics that target the head of the ranked list, making it ideal for surfacing highly relevant, high-confidence recommendations.

2.2.3 Trending Now

This algorithm identifies short-term popularity trends, such as seasonal spikes (e.g., holiday movies in December) or current events (e.g., documentaries during natural disasters). The “Trending Now” row captures these insights and displays them to users in a context-sensitive manner, often combining them with basic personalization.

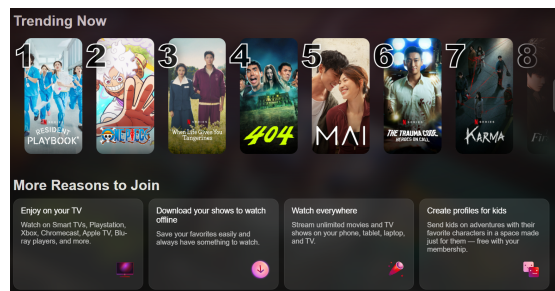


Figure 5: Trending now on Netflix

2.2.4 Continue Watching

This ranker helps users resume previously started content, which is critical for serialized or episodic shows. It takes into account viewing position, time since last session, and whether other titles have been watched in the meantime to prioritize content that the user is most likely to want to return to.

2.2.5 Video-to-Video Similarity (Sims)

This unpersonalized model calculates similarity between videos and powers the “Because You Watched” (BYW) rows. The recommendations are anchored to a specific title and augmented by personalization filters that ensure users do not see content they’ve already watched or are unlikely to enjoy.

2.2.6 Page Generation and Diversity

Once all rows are generated, Netflix employs a page generation algorithm that selects and arranges the rows based on predicted relevance and content diversity. Instead of using a fixed template (as was done prior to 2015), Netflix now constructs the homepage dynamically for each user. This includes deciding which rows to display, their order, and how to best balance novelty, relevance, and serendipity. Some users may see several “Because You Watched” rows, while others may not see any at all, depending on their history and preferences.

This page generation is computationally intensive. With tens of thousands of candidate rows per user, the algorithm must efficiently select a small subset that maximizes engagement. It also tries to mitigate over-specialization by ensuring a mix of content types, themes, and recommendation strategies.

2.2.7 Evidence Selection

Another crucial component of the architecture is the evidence selection engine. For each recommendation, the system selects supporting information—images, summaries, star ratings, cast mentions, awards, or prior user behavior cues—that will be shown alongside the video. The goal is to make recommendations more compelling and contextually meaningful, helping users make faster decisions.

2.2.8 Search-Driven Recommendations

Even the search bar functions as a recommender system. When users search for a title that’s not in the catalog, the system recommends similar or trending content instead. Partial queries are interpreted using personalized search algorithms, transforming search into a recommendation task in its own right.

2.3 Continuous Improvement through Experimentation

Netflix’s recommender system is not static; it evolves continuously through a rigorous experimentation framework. The primary goal is to improve user retention and engagement—key performance indicators for a subscription-based business. Netflix relies heavily on A/B testing, offline experimentation, and iterative algorithm refinement to evaluate and improve its recommendation algorithms.

2.3.1 A/B Testing: The Gold Standard

A/B testing is Netflix’s most trusted method for measuring the real-world impact of algorithmic changes. In these experiments, users are randomly assigned to different test groups (cells), each of which experiences a slightly different version of the recommender system. One group receives the standard (control) experience, while others receive various algorithmic alternatives.

Each test runs for 2 to 6 months, capturing long-term metrics such as:

- **Local engagement:** Are users interacting more with the specific feature or algorithm being tested?
- **Overall engagement:** Are users spending more hours watching content overall?
- **Retention rates:** Are users less likely to cancel their subscription?

2.3.2 Choosing the Right Metrics

Because Netflix's revenue model is based entirely on monthly subscriptions, retention is a top priority. However, since retention improvements can be subtle (often $<0.1\%$), Netflix also tracks proxy metrics like viewing hours and click-through rates to detect meaningful differences.

Metrics are analyzed statistically, and Netflix only accepts a new algorithm if it shows clear improvement across multiple dimensions. Tests with ambiguous or contradictory results are often rerun to verify consistency.

2.3.3 Offline Experimentation: Faster, But Riskier

While A/B tests provide high confidence, they are slow and resource-intensive. To iterate more quickly, Netflix also relies on offline experimentation using historical data. For example, the system can simulate how a new algorithm would have ranked past content, and compare it to what users actually watched.

Offline metrics include:

- Mean reciprocal rank
- Precision and recall
- Catalog coverage and diversity

However, offline experiments assume that user behavior would remain the same even if the new algorithm had been in place, which is not always true. Therefore, these experiments are used primarily for pre-filtering candidate algorithms before launching real A/B tests.

2.3.4 Interpreting A/B Test Results

Despite the statistical rigor, interpreting A/B results often involves a degree of subjective judgment. For instance, an algorithm may improve retention but not increase engagement, raising questions about the actual cause of the improvement. Netflix typically looks for consistent wins across local, global, and retention metrics before making changes permanent.

Netflix also explores variance reduction techniques such as stratified sampling, which makes A/B test cells more comparable by controlling for variables like device type, payment method, or sign-up date.

2.3.5 Testing New vs. Existing Users

Netflix runs tests on both new and existing members, but each group comes with trade-offs. Existing users offer larger sample sizes, but their behavior may be influenced by familiarity with the previous system. New users offer cleaner measurements, since they don't have prior expectations, but their engagement data is limited during the free trial period.

2.3.6 Speeding Up Innovation

To test faster, Netflix sometimes runs multiple A/B tests simultaneously, assigning users to overlapping experiments as long as they don't conflict. It also tests parameter tuning, feature sets, and training methods in parallel to identify the best combinations.

The company has a well-defined innovation pipeline: ideas are first validated offline, then tested online via A/B, and if successful, deployed system-wide. This pipeline ensures that only the most promising ideas reach production, while minimizing user disruption and business risk.



2.4 Conclusions

Netflix's recommender system is a prime example of how advanced algorithms can revolutionize user experience in the digital age. Rather than relying on a single technique, Netflix combines multiple algorithms—such as personalized video ranking, similarity models, trending detection, and contextual evidence selection—to build a highly adaptive and personalized interface. This system is responsible for influencing over 80% of content streamed, highlighting its central role in guiding user engagement.

More than a technical achievement, these algorithms drive real business impact: increasing content discoverability, reducing user churn, and enabling the success of niche titles. By continuously experimenting through A/B testing and refining their models using both offline and real-time data, Netflix ensures that its recommendation engine evolves with user behavior. As personalization becomes more crucial across industries, the Netflix recommender system remains a benchmark for innovation, scale, and effectiveness.

3 Algorithms demonstration

3.1 Content-based filtering: Term Frequency - Inverse Document Frequency (TF-IDF)

Content-based filtering uses item features to recommend other similar items that the user likes, based on their previous actions (click, watch, like) or feedback (review, rating).

TF-IDF's definition is a measure of importance of a word to a document in a collection, adjusted for the fact that some words appear more frequently in general. TF-IDF is the combination of term frequency and inverse document frequency. There are many ways for determining the values of the two statistics.[6]

- Term frequency, $tf(t, d)$, is the frequency of term t within document d . There's various way to calculate it, one common way is to count the number of time t appears divided by the total number of terms in d .

$$tf(t, d) = \frac{f_{t,d}}{\sum_{i \in d} f_{i,d}}$$

- Inverse document frequency indicate how informative the word is, or how common it appear in all documents. It is measured by dividing the total number of documents (N) by the number of documents containing the term (M), then taking the logarithm. The higher IDF is, the more important the term is in distinguishing between documents.

$$idf(t, d) = \log \frac{N}{M}$$

Regarding movie recommendation, movies usually have a lot of textual metadata: Plot summaries, genre, cast, tags, user reviews... Once these textual features are encoded as TF-IDF vectors, you can compute cosine similarity between movies. For example, if two movies share similar plot descriptions, their corresponding vectors will have a higher similarity score, making them likely candidates to be recommended together.

- Cosine similarity is the cosine of the angle between the vectors. It gives a measure of how similar two documents are likely to be, no matter the length of the documents.

$$\cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \cdot \sqrt{\sum_{i=1}^n B_i^2}}$$

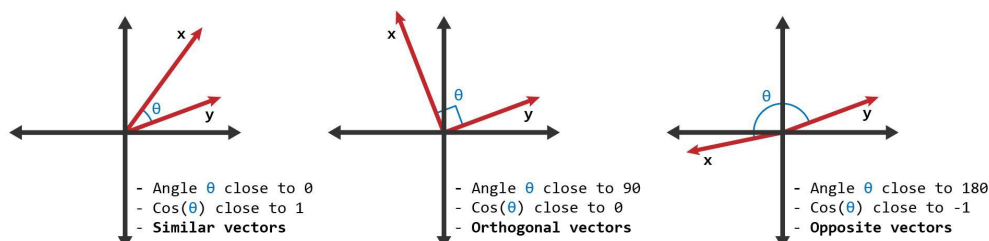


Figure 6: Cosine similarity

3.2 Collaborative filtering: Singular Value Decomposition (SVD)

Collaborative filtering uses similarities between users and items at the same time to recommends. Collaborative filtering models recommend an item to user A based on the interests of a similar user B. The embeddings can be learned automatically, without human hand-engineering of features (latent features).

In collaborative filtering, the matrix representing each user's preference (feedback matrix A) is decomposed into user embedding matrix U and item's feature embedding matrix I using matrix factorization. The matrix A is given, and the matrix U and I are learned. The result matrix will produce a product $\hat{A} = UV^T$ that is a good approximation of the matrix A , and we use this matrix $\hat{A}$ to predict the users' ratings for the remaining items in order to rank and recommend them.



Figure 7: Matrix factorization

Singular Value Decomposition (SVD) is a tool that can help with this task. In linear algebra, SVD is a factorization of a matrix into a rotation, followed by a rescaling followed by another rotation with the form $M = U\Sigma V^T$. Because SVD is a technique that can break a matrix into many matrixes, it is commonly used in recommender system.[5]

The process of performing SVD on the feedback matrix includes three steps:[3]

1. Factorize the feedback matrix into three matrices: U , S , and V . Matrix U represents the users, with each row is a user and each column is a latent feature or dimension. The latent features are determined by the singular values in matrix S . Matrix V represents the items, where each row is an item and each column is a latent feature.
2. Keep only the top k singular values and their corresponding vectors to reduce computational costs without losing too much information.
3. Take the product of those matrices back together USV^T to get an approximation of the original feedback matrix.

We have to choose an objective function to evaluate how well the model predict. One simple function is the sum of square error (comparing predict value to the original matrix):

$$\min_{U \in \mathbb{R}^{m \times d}, V \in \mathbb{R}^{n \times d}} \sum_{(i,j) \in \text{obs}} (A_{ij} - \langle U_i, V_j \rangle)^2.$$

Some common algorithms to minimize the cost function:

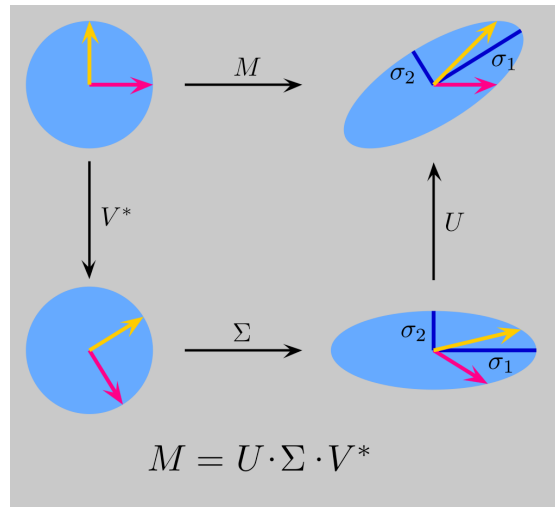


Figure 8: SVD in linear algebra, visualized

- Stochastic gradient descent (SGD): A general method to minimize loss functions. Regular gradient descent uses the entire dataset to calculate the gradient, but SGD updates the parameters using only one data point at a time (a subset of dataset). This makes it faster for large datasets, but it can be noisier and less stable.
- Alternating Least Squares (ALS): A specialized method suitable for the mentioned objective function. Since the loss function $q_i^T p_u$ is non-convex, gradient descent method will learn slower and has expensive computations. The basic idea of ALS is to learn one of q and p at a time for optimization while keeping the other as constant. This makes the objective at each iteration convex and solvable. The alternating between q and p stops when there is convergence to the optimal. This computation can be parallelised which makes the algorithm more efficient when the dataset is large and the user-item rating matrix is super sparse (typical recommendation scenarios).

4 Algorithm implementation and result

In this section, we use the MovieLens datasets to implement these models. The MovieLens dataset is available in various sizes: 100k, 1m, 10m, and 20m. In our implementation, we use the 100k dataset as the primary source (including user data and movie data). The implements of 2 algorithms is pushed to Github¹ too.

4.1 Term Frequency - Inverse Document Frequency Model Implement

First, we import the necessary libraries to perform calculations and load the dataset. 'TfidfRecommender' is a custom utility for cleaning, tokenizing, and fitting the data.

```
1 import sys
2
3 from recommenders.datasets import movielens
4 from recommenders.models.tfidf.tfidf_utils import TfidfRecommender
5 from recommenders.datasets.python_splitters import python_random_split
6 from recommenders.evaluation.python_evaluation import (
7     map_at_k,
8     ndcg_at_k,
9     precision_at_k,
10    recall_at_k,
11 )
12 import pandas as pd
13 import numpy as np
14 from sklearn.metrics.pairwise import cosine_similarity
15 from sklearn.preprocessing import normalize
```

To use the MovieLens dataset, we need to download it from GroupLens² and load it into a pandas DataFrame. The 'load_item_df()' function helps us do this, with additional options such as adding the 'movieID' and 'title' columns.

These variables define the number of top recommendations to generate ('TOP_K'), the specific user to generate recommendations for ('user_id'), and the size of the MovieLens dataset to use ('MOVIELENS_DATA_SIZE').

```
1 # Top k items to recommend
2 TOP_K = 10
3 # User ID to recommend for
4 user_id = 164
5 # Select MovieLens data size: 100k, 1m, 10m, or 20m
6 MOVIELENS_DATA_SIZE = "100k"
7 # Load movies dataset into a dataframe
```

Let's begin by loading the metadata file for the dataset into a Pandas dataframe. We load dataframe for user and items (movies). The code below loads the movie metadata (movie ID, title, genres) into a Pandas DataFrame using the 'movielens.load_item_df()' function from the Recommenders library. This dataset is used for TF-IDF model learn about movies.

```
1 item_data = movielens.load_item_df(
2     size=MOVIELENS_DATA_SIZE,
3     local_cache_path=None,
4     movie_col="movieID",
5     title_col="title",
6     genres_col="genres"
7 )
8
9 # item_data.head()
```

¹<https://github.com/HuynhNhan27/Recommender-System>

²<https://grouplens.org/datasets/movielens/>

This code loads the user interaction data (user ID, item ID, rating) into another Pandas DataFrame. This dataset is used for prediction and evaluation later.

```
1 # Load user interaction data
2 user_data = movielens.load_pandas_df(
3     size=MOVIELENS_DATA_SIZE, header=["userID", "itemID", "rating"]
4 )
5 # user_data.head()
```

The user interaction data is then split into training and testing sets using ‘python_random_split’ to evaluate the model’s performance on unseen data.

```
1 # Split user data into train and test sets
2 train_user_data, test_user_data = python_random_split(user_data, 0.75)
```

Here, we initialize the ‘TfidfRecommender’ model. The ‘tokenization_method="none"’ indicates that we’ll treat the genres string as a single document for TF-IDF. The ‘clean_dataframe’ function preprocesses the ‘genres’ column.

```
1 recommender = TfidfRecommender(
2     id_col='movieID',
3     tokenization_method="none"
4 ) # Create the recommender object
5 cols_to_clean = ['genres']
6 clean_col = 'cleaned_genres'
7 item_data_clean = recommender.clean_dataframe(
8     item_data.copy(),
9     cols_to_clean, clean_col
10 ) # Clean dataframe
```

The ‘tokenize_text’ function calculates the TF-IDF matrix based on the cleaned genres. The ‘fit’ function trains the TF-IDF vectorizer.

```
1 tf, vectors_tokenized = recommender.tokenize_text(
2     item_data_clean,
3     text_col=clean_col
4 ) # Tokenize text
5 # Fit the TF-IDF vectorizer (calculate TF-IDF matrix)
6 recommender.fit(tf, vectors_tokenized)
```

To evaluate TF-IDF Model, we define two function to recommend and evaluate that recommendation:

- Function ‘recommend_movies_for_user’ takes a list of liked movie IDs, seen movie IDs, the movie DataFrame, and the TF-IDF matrix as input. It calculates a user profile based on the average TF-IDF vector of the liked movies and then finds the top ‘total_recommend’ movies with the highest cosine similarity to this user profile, excluding the movies the user has already seen. Recommendations are grouped by similarity for diversity.
- The ‘evaluate_tfidf’ function iterates through each unique user in the test set, retrieves their liked and seen items from the training data, generates recommendations using ‘recommend_movies_for_user’, and then evaluates the recommendation quality using metrics like MAP@K, NDCG@K, Precision@K, and Recall@K. The similarity score from the cosine similarity is used as the prediction score.

Because that functions are pretty long, we don’t show that functions in here. You can see it in my Github repository. The model is ready, and now we will test it: Recommend for user with id 164. This code demonstrates how to get recommendations for a specific ‘user_id’ (164 in this case). It retrieves the user’s liked and seen movies from the test set and then calls ‘recommend_movies_for_user’ to get the top recommendations.

```

1 # Predict for a specific user
2 user_id = 164
3 # Get the user history from the test set
4 user_history = test_user_data[test_user_data['userID'] == user_id]
5 liked_items = user_history[user_history['rating'] > 3]['itemID'].tolist()
6 seen_items = user_history['itemID'].tolist()
7 # Recommend movies for the user
8 recommendations_df = recommend_movies_for_user(
9     liked_items,
10    seen_items,
11    item_data_clean,
12    recommender.tfidf_matrix,
13    total_recommend=TOP_K,
14 )
15
16 recommendations_df

```

Below is the list of the top 10 movies recommended for user 164 by the TF-IDF model. As you can see, all of them are thriller movies. The model has learned that thriller is the user's preferred genre and recommends unseen movies within the same category.

	movieID	title	genres	similarity
0	1042	Just Cause (1995)	Mystery Thriller	0.683923
1	505	Dial M for Murder (1954)	Mystery Thriller	0.683923
2	144	Die Hard (1988)	Action Thriller	0.550679
3	265	Hunt for Red October, The (1990)	Action Thriller	0.550679
4	348	Desperate Measures (1998)	Crime Drama Thriller	0.517858
5	332	Kiss the Girls (1997)	Crime Drama Thriller	0.517858
6	1624	Hush (1998)	Thriller	0.509571
7	987	Underworld (1997)	Thriller	0.509571
8	914	Wild Things (1998)	Crime Drama Mystery Thriller	0.486161
9	696	City Hall (1996)	Drama Thriller	0.478265

Figure 9: Recommendation for user 164 using TF-IDF

Finally, the 'evaluate_tfidf' function is called to assess the overall performance of the TF-IDF model on the test data.

```

1 # Evaluate the model
2 evaluate_tfidf(test_user_data, item_data_clean, recommender, top_k=TOP_K)

```

The result is very surprise: scores of MAP, NDCG, Precision, Recall are all zero. We will discuss about that scores later.

4.2 Singular Value Decomposition Model Implement

For the SVD model, we import the necessary libraries for model training and dataset loading (similar to the TF-IDF setup).

```

1 import sys
2 import surprise
3
4 from recommenders.utils.timer import Timer
5 from recommenders.datasets import movielens

```

```
6 from recommenders.datasets.python_splitters import python_random_split
7 from recommenders.evaluation.python_evaluation import (
8     rmse,
9     mae,
10    rsquared,
11    exp_var,
12    map_at_k,
13    ndcg_at_k,
14    precision_at_k,
15    recall_at_k,
16    get_top_k_items,
17 )
18 from recommenders.models.surprise.surprise_utils import (
19     predict,
20     compute_ranking_predictions,
21 )
22 from recommenders.utils.notebook_utils import store_metadata
```

Key variables are same with TF-IDF model, so we don't repeat it here. We continue to use the MovieLens 100k dataset to allow for comparison with the TF-IDF model. Unlike TF-IDF, SVD is a collaborative filtering model, so it only requires user-related data, such as user IDs and the movie IDs of rated movies.

```
1 # Select MovieLens data size: 100k, 1m, 10m, or 20m
2 MOVIELENS_DATA_SIZE = "100k"
3 # Load user dataset into a dataframe
4 data = movielens.load_pandas_df(
5     size=MOVIELENS_DATA_SIZE,
6     header=["userID", "itemID", "rating"],
7     local_cache_path=None
8 )
9 data.head()
```

The loaded data is then split into training and testing sets using 'python_random_split' from the Recommenders library. The training set is further converted into a Surprise 'Trainset' object, which is required by the Surprise library's algorithms.

```
1 # Prepare the data for training
2 train, test = python_random_split(data, 0.75)
3 train_set = surprise.Dataset.load_from_df(
4     train, reader=surprise.Reader("ml-100k")
5 ).build_full_trainset()
```

Here, an SVD model from the Surprise library is initialized with specific parameters: 'random_state' for reproducibility, 'n_factors' controlling the dimensionality of the latent factor space, 'n_epochs' specifying the number of training iterations, and 'verbose' to display training progress. The model is then trained on the 'train_set', and the training time is recorded using the 'Timer' utility.

```
1 svd = surprise.SVD(
2     random_state=0, n_factors=200,
3     n_epochs=30, verbose=True)
4 with Timer() as train_time:
5     svd.fit(train_set)
6
7 print(f"Took {train_time.interval} seconds for training.")
```

This code generates predictions on the 'test' set using the trained SVD model. It also computes ranking predictions for all items for the users in the training set, excluding items they have already interacted with. This is used for evaluating ranking metrics later. The prediction time is also recorded.

```

1 # Predictions on the test set
2 predictions = predict(
3     svd, test, usercol="userID",
4     itemcol="itemID"
5 )
6 # Remove seen items from the test set
7 with Timer() as test_time:
8     all_predictions = compute_ranking_predictions(
9         svd, train, usercol="userID",
10        itemcol="itemID", remove_seen=True
11    )
12 print(f"Took {test_time.interval} seconds for predictions.")

```

Finally, we extract the top 10 movie recommendations for user ID 164. Unlike the TF-IDF model, the SVD model already incorporates user data, so it doesn't require separate user vector calculations. However, since the SVD model does not include movie metadata, the recommendation output only provides movie IDs, which can be hard for users to interpret.

```

1 # Get the top k items for user
2 get_top_k_items(
3     all_predictions[all_predictions["userID"] == 164],
4     col_rating="prediction", k=20
5 )

```

	userID	itemID	prediction	rank
0	164	50	5.0	1
1	164	126	5.0	2
2	164	134	5.0	3
3	164	169	5.0	4
4	164	189	5.0	5
5	164	207	5.0	6
6	164	408	5.0	7
7	164	430	5.0	8
8	164	480	5.0	9
9	164	483	5.0	10

Figure 10: Recommendation for user 164 using SVD

After checking the recommended movie IDs, the results are quite promising. The SVD model suggests some well-known movies such as *Star Wars*, *Citizen Kane*, and *The Wrong Trousers*.

Below part evaluates the performance of the trained SVD model using various regression metrics (RMSE, MAE, R-squared, Explained Variance) on the 'predictions' DataFrame and ranking metrics (MAP@K, NDCG@K, Precision@K, Recall@K) on the 'all_predictions' DataFrame. These metrics quantify the accuracy and ranking quality of the recommendations.

```

1 # Evaluation metrics
2 eval_rmse = rmse(test, predictions)
3 eval_mae = mae(test, predictions)
4 eval_rsquared = rsquared(test, predictions)
5 eval_exp_var = exp_var(test, predictions)
6

```



```
7 eval_map = map_at_k(test, all_predictions, col_prediction="prediction", k=TOP_K)
8 eval_ndcg = ndcg_at_k(test, all_predictions, col_prediction="prediction", k=TOP_K)
9 eval_precision = precision_at_k(
10     test, all_predictions, col_prediction="prediction", k=TOP_K
11 )
12 eval_recall = recall_at_k(test, all_predictions, col_prediction="prediction", k=
    TOP_K)
13
14 print(
15     "RMSE:\t\t%f" % eval_rmse,
16     "MAE:\t\t%f" % eval_mae,
17     "rsquared:\t%f" % eval_rsquared,
18     "exp var:\t%f" % eval_exp_var,
19     sep="\n",
20 )
21 print("----")
22 print(
23     "MAP:\t\t%f" % eval_map,
24     "NDCG:\t\t%f" % eval_ndcg,
25     "Precision@K:\t%f" % eval_precision,
26     "Recall@K:\t%f" % eval_recall,
27     sep="\n",
28 )
```

```
RMSE:          0.948771
MAE:           0.747003
rsquared:      0.288045
exp var:       0.288157
----
MAP:           0.051213
NDCG:          0.109878
Precision@K:   0.100318
Recall@K:      0.035359
```

Figure 11: Evaluation of SVD model

To know what Machine Learning improves in SVD Model, we will implement another baseline SVD, and then compare it with our custom model. The comparison of 2 models will be discussed in next subsection.

4.3 Evaluate TF-IDF and SVD model

The evaluation of the TF-IDF content-based recommender system yielded scores of 0 for MAP@K, NDCG@K, Precision@K, and Recall@K. This suggests that the model, in its current configuration, is not effectively recommending movies that align with the positive interactions (likes) observed in the test dataset. Several factors likely contribute to this outcome:

- **Limitations of Content Representation (Insufficient Granularity):** As you correctly pointed out, the content representation is solely based on the 'genres' of the movies. With only 19 distinct genres available in the Movielens dataset, the granularity of movie descriptions is quite coarse. Many movies share the same genre classifications, potentially leading to high similarity scores between movies that users might perceive as quite different. This lack of nuanced content features makes it challenging for the TF-IDF model to accurately capture

the subtle differences in movie characteristics that drive individual user preferences. Consequently, the recommendations generated based on broad genre overlap may not resonate with specific user tastes.

- **Disconnect Between "Seen" and "Favorite" (Weak Signal in User History):** Your observation that user interactions might be heavily influenced by factors other than genuine preference (e.g., popularity, trends) is crucial. If a significant portion of the movies users have seen and rated positively were driven by external factors rather than intrinsic enjoyment of the movie's content, then using these "likes" to build a genre-based user profile becomes less reliable. The model is learning preferences based on what users watched and rated favorably for various reasons, not necessarily what they truly favor in terms of underlying content. This introduces noise into the user profiles and hinders the model's ability to recommend genuinely relevant content.
- **Potential Use Case: "Similar to What You've Seen" Section:** Despite the poor performance in a general recommendation setting, your insight about the TF-IDF model's tendency to recommend movies with similar genres highlights a potential niche application: a "Similar to What You've Seen" section. In this context, the model's strength in identifying movies with overlapping genre classifications could be valuable for users looking for more of what they have already experienced, regardless of the underlying reasons for their initial viewing. This feature could cater to users who enjoyed a movie for its genre and are seeking similar experiences.

This is evaluation score of custom SVD and baseline SVD model. There are 8 evaluation score and according to that, we figure that 2 models have the score of basic level recommendation quality:

- While the Baseline SVD exhibits slightly better performance in terms of rating prediction accuracy (lower RMSE and MAE, higher R-squared and explained variance), the Custom SVD shows a slight advantage in ranking quality, particularly as indicated by the higher NDCG@K and Recall@K. The MAP@K and Precision@K are quite similar between the two models.
- Modifications made in the Custom SVD model (SVD + ML) might have slightly shifted the model's focus towards better ranking of relevant items, potentially at the cost of a minor decrease in pure rating prediction accuracy.
- The choice of which model is "better" would depend on the specific goals of the recommender system. If accurate rating prediction is paramount, the Baseline SVD might be preferred. However, if the primary goal is to provide well-ranked and relevant recommendations, the Custom SVD appears to be slightly more effective. The differences, however, are relatively small, indicating that both the standard SVD and the customized version perform comparably on this dataset.

With the result above, we can compare two model and get some conclusion about two model:

- **TF-IDF Model:**
 - Use cases:
 - * Text classification.
 - * Content-based recommendation (e.g., movie or article recommender systems).

- Strengths:
 - * Simple and interpretable.
 - * Effective for capturing term relevance.
 - * Not get cold-start issue.
- Limitations:
 - * Cannot capture semantic similarity (e.g., synonyms).
 - * Cannot recommend new favorite for user.
 - * High-dimensional and sparse.
- SVD Model:
 - Use cases:
 - * Dimensionality reduction.
 - * Topic modeling.
 - * Collaborative filtering (e.g., matrix factorization for user-item ratings).
 - Strengths:
 - * Captures hidden (latent) relationships.
 - * Reduces noise and redundancy.
 - * Useful for discovering similarity even when terms don't overlap.
 - Limitations:
 - * Less interpretable (latent dimensions are abstract).
 - * Cold-start issue.
 - * Computationally more expensive.

References

- [1] Paul Covington, Jay Adams, and Emre Sargin. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems*, RecSys '16, page 191–198, New York, NY, USA, 2016. Association for Computing Machinery.
- [2] Carlos A. Gomez-Uribe and Neil Hunt. The netflix recommender system: Algorithms, business value, and innovation. *ACM Trans. Manage. Inf. Syst.*, 6(4), December 2016.
- [3] Ritik Gupta. How singular value decomposition (svd) is used in recommendation systems, "clearly explained". https://medium.com/@ritik_gupta/how-singular-value-decomposition-svd-is-used-in-recommendation-systems-clearly-explained-201b, 2023. Accessed: 2025-04-14.
- [4] Caballar & Stryker. What is a recommendation engine. 2024.
- [5] Adrian Tam. Using singular value decomposition to build a recommender system. <https://machinelearningmastery.com/using-singular-value-decomposition-to-build-a-recommender-system/>, 2021. Accessed: 2025-04-14.
- [6] Ramzan Zdemir. Recommendation systems: Content-based (tf-idf). <https://www.kaggle.com/code/ramzanzdemir/recommendation-systems-content-based-tf-idf>, 2023. Accessed: 2025-04-14.